Test-Based Verification of LLM-Generated Code in Multi-Step Repository Tasks

Abdulselam Hussen

Department of Computer Science, University of Nairobi, Nairobi, Kenya

March 2024

Abstract

Agentic coding systems are commonly evaluated by whether a repository's visible test suite passes after the model's changes. We argue that this signal overstates correctness in multi-step repository tasks, where requirements span several milestones and include unstated edge cases. We propose a four-layer verification protocol (visible tests, hidden edge-case tests, regression checks, and artifact checks) and apply it to 900 trajectories produced from 60 tasks, three anonymised models, and five runs each. On this fictional dataset, visible tests alone report pass rates of 64.3–78.3%, while 41.0–52.3% of runs survive all four layers, an overstatement of 23.3–27.0 percentage points. Hidden edge-case tests account for 150 of the 229 additional failures detected. Two reviewers applying a binary rubric reached Cohen's $\kappa = 0.84$, compared with 0.61 when using free-form notes. We discuss implications for evaluation practice and the limits of this study.

Keywords: code generation, agentic evaluation, test-based verification, software testing, rubric design

1 Introduction

Large language models are increasingly used as coding agents that read a repository, run commands, and edit files over many steps. Evaluating such agents requires deciding when a change is *correct*. The prevailing practice is to run the tests that ship with the repository (or with the benchmark) and report the share of runs that pass.

This practice has a weakness. Visible tests encode what the task author *thought to check*, and an agent can satisfy them while missing requirements that were stated in prose, ignoring constraints, or breaking behaviour elsewhere in the code base. In multi-step tasks the gap widens, because each milestone adds requirements that a single test file rarely covers.

This paper makes three contributions:

1. a layered verification protocol that separates failure types (Section 3);
2. an empirical comparison of visible-test pass rates and fully verified pass rates on a sample set of 900 trajectories (Section 5);
3. a binary rubric and reviewer protocol, with an agreement analysis (Section 5).

2 Related Work

Early code-generation benchmarks evaluate short, self-contained functions against hidden unit tests [1, 2]. Later work moved to repository-level problems: SWE-bench asks a model to resolve real GitHub issues and checks the result with tests from the originating pull request [3]. Liu et

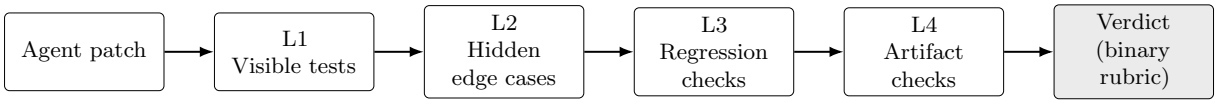


Figure 1: Verification pipeline. A run must pass each layer in order to be counted as fully verified.

al. showed that the tests in widely used benchmarks are often too weak, and that augmenting them lowers measured pass rates substantially [4].

We build on this line of work in two ways. First, we treat edge-case strength as one layer among several rather than a single fix. Second, we add regression and artifact layers, which matter in multi-step tasks where the final repository state, not only a single function, is the object being judged.

3 Method

3.1 Task construction

Each task consists of (i) a user-intent statement written in plain language, (ii) three to five milestones, (iii) acceptance criteria per milestone, (iv) a set of hidden edge cases, and (v) a reference solution. Tasks are run inside an isolated container holding the repository, pinned dependencies, and the visible test suite, so that every run starts from an identical state.

3.2 Verification layers

A run is evaluated by four layers applied in order. Let $v_j(r) \in \{0, 1\}$ indicate whether run r passes layer j.

L1: Visible tests.
The test suite supplied with the repository.

L2: Hidden edge-case tests.
Author-written tests for boundary values, malformed input, and error handling that were not shown to the agent.

L3: Regression checks.
The full pre-existing suite, plus differential checks on modules the task should not have changed.

L4: Artifact checks.
Verification of non-code requirements stated in the task, such as migrations, documentation updates, and configuration constraints.

The cumulative pass rate at layer k over N runs is

$$P_k = \frac{1}{N} \sum_{i=1}^{N} \prod_{j=1}^{k} v_j(r_i), \qquad k = 1, \ldots, 4, \tag{1}$$

and the *overstatement* is $P_1 - P_4$.

3.3 Rubric and reviewer protocol

Final verdicts follow a binary rubric with one line per requirement ("met" or "not met") and a required evidence field citing a command, log line, or test result. Reviewers are blind to the model that produced each trajectory. Two reviewers independently graded a random subset of 120 trajectories to measure agreement.

Table 1: Cumulative pass rate by verification layer (fictional data; $N = 300$ runs per model; counts in parentheses).

Model	L1	L1–L2	L1–L3	L1–L4
A	78.3% (235)	62.0% (186)	55.7% (167)	52.3% (157)
B	71.0% (213)	52.7% (158)	47.3% (142)	44.0% (132)
C	64.3% (193)	49.0% (147)	42.7% (128)	41.0% (123)

Table 2: Runs that passed L1 but failed a later layer (fictional data).

Layer of first failure	Model A	Model B	Model C	Total
L2: hidden edge cases	49	55	46	150
L3: regressions	19	16	19	54
L4: artifacts	10	10	5	25
Total	78	81	70	229

4 Experimental Setup

We constructed 60 tasks (45 Python, 8 Bash, 7 SQL) over repositories of 1.2k–18k lines of code. Each task has 4–12 hidden edge-case tests. Three anonymised models (A, B, C) each attempted every task five times, giving $N = 300$ runs per model and 900 in total. Runs had a 45-minute limit and a fixed container image. No model received the hidden tests, the rubric, or the reference solution.

5 Results

5.1 Pass rates by layer

Table 1 shows cumulative pass rates. Visible tests alone gave pass rates of 64.3–78.3%. After all four layers, 41.0–52.3% of runs remained. The overstatement was 26.0 points for Model A, 27.0 for Model B, and 23.3 for Model C. The ordering of the models did not change across layers.

5.2 Where the extra failures come from

Of the 641 runs that passed L1, 229 failed a later layer (Table 2). Hidden edge-case tests caught most of them (150 runs, 65.5%), followed by regression checks (54 runs, 23.6%) and artifact checks (25 runs, 10.9%). Typical L2 failures were unhandled empty input and off-by-one boundaries; typical L4 failures were missing migration files and ignored configuration constraints.

5.3 Reviewer agreement

On the 120 doubly graded trajectories, reviewers using the binary rubric with an evidence field reached Cohen's $\kappa = 0.84$. Using free-form notes on a comparable subset gave $\kappa = 0.61$. Most disagreements under free-form notes concerned whether a partially met requirement should count as a pass.

6 Discussion

Implications. Reporting only visible-test pass rates would have overstated every model by more than 23 percentage points in this sample. Reporting each layer separately also tells a developer *why* a run failed, which a single number cannot do.

Threats to validity. The task set is small and the models are anonymised, so results may not generalise. Hidden tests are written by the same people who write tasks, which may bias what counts as an edge case. Reviewers may tire over long sessions, and the binary rubric trades nuance for consistency. A real study would add confidence intervals, a pre-registered analysis plan, and an independent replication.

Future work. Extending the protocol to additional languages, adding performance and security layers, and studying how much hidden-test authorship affects measured pass rates.

7 Conclusion

Visible-test pass rates overstate the correctness of agent-generated changes in multi-step repository tasks. A layered protocol with a binary, evidence-backed rubric exposes where and why runs fail, and improves reviewer agreement. In this sample, the largest share of hidden failures came from edge cases rather than regressions or missing artifacts.

Acknowledgements

The author would like to express sincere gratitude to the staff and members of the Department of Computer Science at the University of Nairobi for their unwavering support throughout this work.

References

[1] M. Chen et al. Evaluating large language models trained on code. *arXiv:2107.03374*, 2021.

[2] J. Austin et al. Program synthesis with large language models. *arXiv:2108.07732*, 2021.

[3] C. E. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[4] J. Liu, C. S. Xia, Y. Wang, and L. Zhang. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.